

BRAC^t's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 1

Roll no: 32 PRN No: 12414914 Name of Student: **Rashi Gundawar**

Subject Name : Deep Learning

Problem Statement : Install and configure TensorFlow/Keras in Google Colab. Perform data preprocessing, normalization, train-test split, and visualization on a sample dataset.

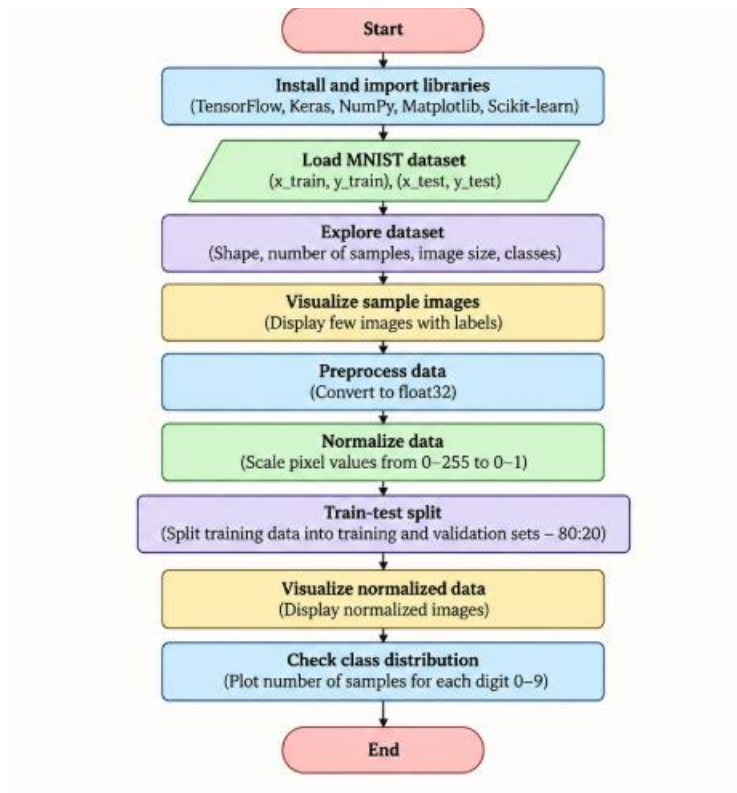
Algorithm :

1. Start
2. Import Required Libraries
Import TensorFlow, Keras, NumPy, Matplotlib, and Scikit-learn.
3. Load MNIST Dataset
Load the MNIST handwritten digit dataset using Keras.
4. Explore the Dataset
Check the number of samples, image size, classes, and pixel value range.
5. Visualize Sample Images
Display sample MNIST images along with their corresponding labels.
6. Preprocess the Data
Convert the image pixel values to float32 data type.
7. Normalize the Data
Scale pixel values from the range 0–255 to 0–1 by dividing by 255.
8. Split the Dataset
Divide the training data into 80% training data and 20% validation data.
9. Visualize Normalized Data
Display sample images after normalization.
10. Analyze Class Distribution
Plot the number of images available for each digit class from 0 to 9.

11. Display Dataset Summary

Display the final shapes, pixel value range, and number of classes.

12. End



Code :

```
import tensorflow as tf
from tensorflow import keras
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Load MNIST dataset
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

print("Dataset loaded successfully!")
print("Training images:", x_train.shape)
print("Training labels:", y_train.shape)
print("Testing images:", x_test.shape)
print("Testing labels:", y_test.shape)

Dataset loaded successfully!
Training images: (60000, 28, 28)
Training labels: (60000,)
Testing images: (10000, 28, 28)
Testing labels: (10000,)

## Display basic information

print("Number of training samples:", len(x_train))
print("Number of testing samples:", len(x_test))
print("Image size:", x_train[0].shape)
print("Number of classes:", len(np.unique(y_train)))
print("Classes:", np.unique(y_train))

Number of training samples: 60000
Number of testing samples: 10000
Image size: (28, 28)
Number of classes: 10
Classes: [0 1 2 3 4 5 6 7 8 9]
```

```
print("Minimum pixel value:", x_train.min())
print("Maximum pixel value:", x_train.max())
```

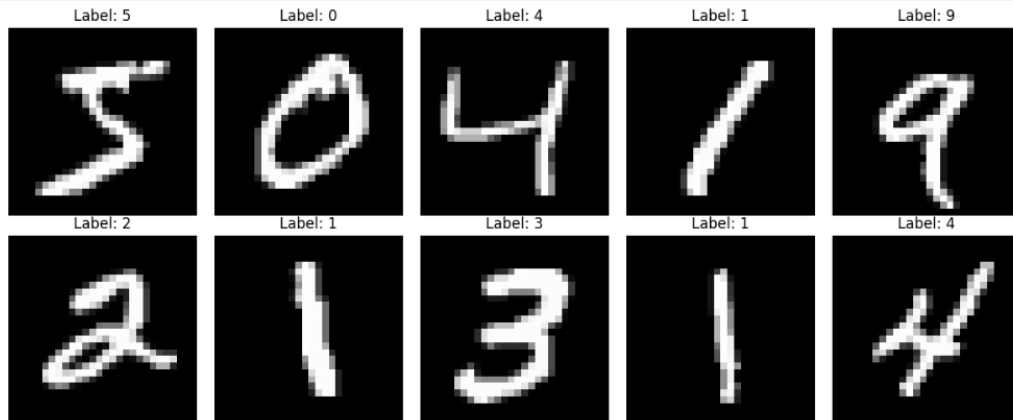
```
... Minimum pixel value: 0
Maximum pixel value: 255
```

```
# Display 10 sample images
```

```
plt.figure(figsize=(12, 5))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train[i], cmap="gray")
    plt.title(f"Label: {y_train[i]}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```



```
# Convert pixel values to float32
```

```
x_train = x_train.astype("float32")
x_test = x_test.astype("float32")

print("Data type after conversion:", x_train.dtype)
```

```
Data type after conversion: float32
```

```
# Normalize pixel values
```

```
x_train = x_train / 255.0
x_test = x_test / 255.0

print("Minimum value after normalization:", x_train.min())
print("Maximum value after normalization:", x_train.max())
```

```
Minimum value after normalization: 0.0
Maximum value after normalization: 1.0
```

```
from sklearn.model_selection import train_test_split

# Split training data into training and validation sets
```

```
x_train, x_val, y_train, y_val = train_test_split(
    x_train,
    y_train,
    test_size=0.2,
    random_state=42,
    stratify=y_train
)
```

```
print("Training samples:", x_train.shape[0])
print("Validation samples:", x_val.shape[0])
print("Testing samples:", x_test.shape[0])
```

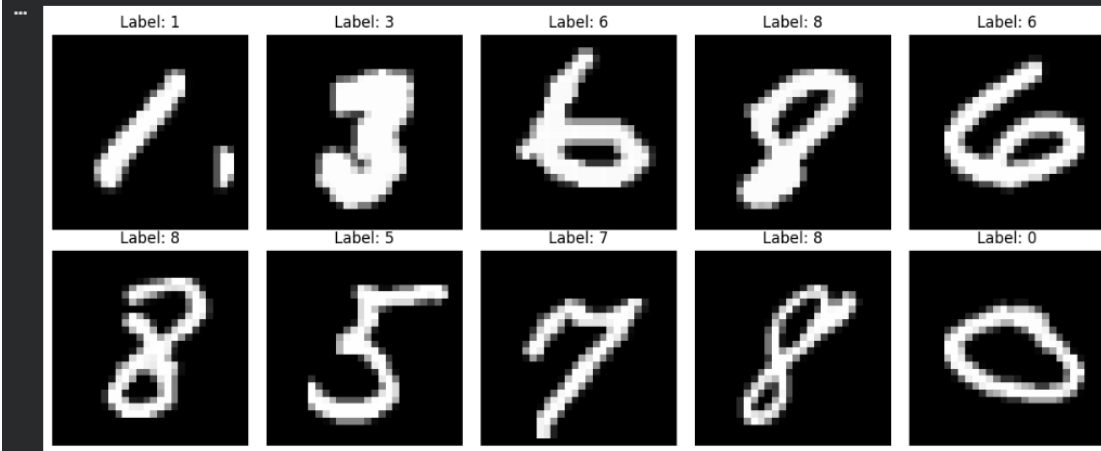
```
Training samples: 48000
Validation samples: 12000
Testing samples: 10000
```

```
# Display normalized images

plt.figure(figsize=(12, 5))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train[i], cmap="gray")
    plt.title(f"Label: {y_train[i]}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```



```
# Count samples for each digit

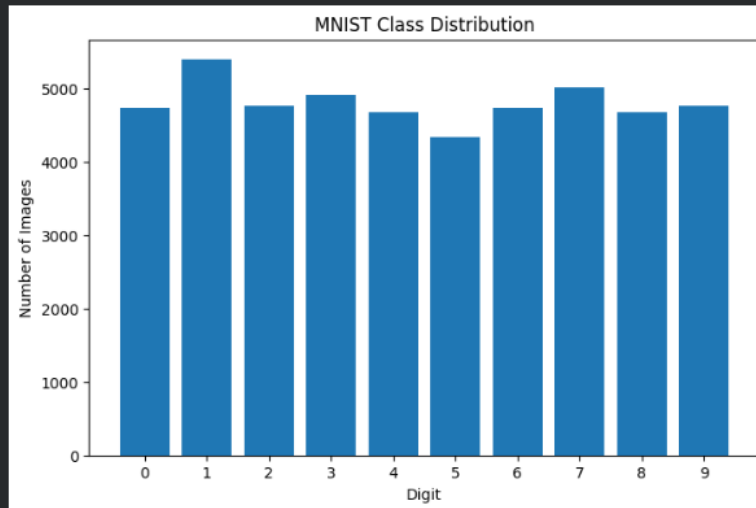
classes, counts = np.unique(y_train, return_counts=True)

plt.figure(figsize=(8, 5))

plt.bar(classes, counts)

plt.xlabel("Digit")
plt.ylabel("Number of Images")
plt.title("MNIST Class Distribution")
plt.xticks(classes)

plt.show()
```



```
# Display final dataset information

print("=" * 45)
print("      MNIST DATASET SUMMARY")
print("=" * 45)

print("Training data shape  :", x_train.shape)
print("Validation data shape :", x_val.shape)
print("Testing data shape   :", x_test.shape)

print("\nPixel value range after normalization:")
print("Minimum:", x_train.min())
print("Maximum:", x_train.max())

print("\nNumber of classes:", len(np.unique(y_train)))
print("Classes:", np.unique(y_train))
```

```
=====
      MNIST DATASET SUMMARY
=====
Training data shape  : (48000, 28, 28)
Validation data shape : (12000, 28, 28)
Testing data shape   : (10000, 28, 28)

Pixel value range after normalization:
Minimum: 0.0
Maximum: 1.0

Number of classes: 10
Classes: [0 1 2 3 4 5 6 7 8 9]
```